

UNIVERSITY OF PADOVA

School of Engineering
Department of Information Engineering



Technical Report for
**Quantum Information and Computing
Laboratory Activities**

by

Maximilian Pries - 2144162

Submitted to

Prof. Dr. Giuseppe Vallone

Padova, June 8, 2025

1 CHSH Inequality Violation

In 1935, Einstein, Podolsky, and Rosen published an argument on the incompleteness of quantum mechanics and proposed the existence of a local hidden variable theory (LHVT) that governs the behavior of particles [1]. In 1964, however, J. S. Bell proved the incompatibility of this LHVT with statistical predictions of quantum mechanics [2]. The goal of this laboratory activity is to generate and measure entangled photon pairs in order to test this incompatibility by experimentally violating the CHSH inequality, and providing evidence against LHVTs.

1.1 Experimental Setup

The hardware setup for the experiment is schematically depicted in Figure 1.1. The main elements of interest are:

- a strong laser (or pump) emitting horizontally polarized photons at 405 nm wavelength,
- a half-waveplate (HWP) to change the polarization of the photons from horizontal to diagonal ($|D\rangle$) or antidiagonal ($|A\rangle$),
- two non-linear crystals, which convert one high-energy photon into a pair of low-energy photons at 810 nm, and emit them on two coaxial cones in a stochastic process,
- two walk-off crystals (WO), which refract the photons of the inner cone, making the cones indistinguishable,
- two quarter-waveplates (QWP) and two HWPs to change the polarization of the photon pairs (in this experiment, only the HWPs are used),
- two polarizing beam splitters (PBS), that only transmit horizontally polarized photons,
- two accumulators and sensors in Geiger mode, which fire when some photons are detected.

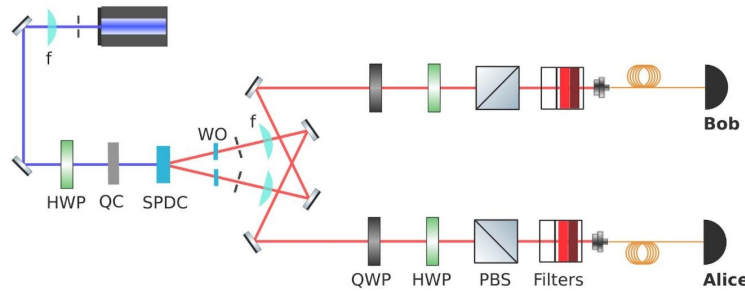


Figure 1.1: Hardware setup for the laboratory experiments.

To distinguish the elements on the split path, they are denoted by A and B . After the WOs, the two photons are entangled and can be described by their joint state $|\Psi\rangle$:

$$|\Psi\rangle = \frac{1}{\sqrt{2}}(|HH\rangle \pm |VV\rangle) = |\Phi^\pm\rangle \quad (1.1)$$

For the violation of the CHSH inequality, the CHSH parameter δ_{CHSH} has to be calculated and be > 2 . To compute the parameter, the correlations between the two photons in two different bases for A and B are needed. The measurement basis for each of the photons can be set by changing the angles of the HWPs. In one experiment, only one of the outcomes for a basis can be measured. The chosen measurement operators and bases are:

$$\hat{A}_0 : \sigma_z \rightarrow \{0 : |H\rangle, 1 : |V\rangle\} \quad (1.2)$$

$$\hat{A}_1 : \sigma_x \rightarrow \{0 : |D\rangle, 1 : |A\rangle\} \quad (1.3)$$

$$\hat{B}_0 : \frac{1}{\sqrt{2}}(\sigma_x + \sigma_z) \rightarrow \left\{0/1 : \cos\theta |H\rangle + \sin\theta |V\rangle, \text{ where } \theta = \frac{\pi}{8}, \frac{3\pi}{8}\right\} \quad (1.4)$$

$$\hat{B}_1 : \frac{1}{\sqrt{2}}(\sigma_x - \sigma_z) \rightarrow \left\{0/1 : \cos\theta |H\rangle + \sin\theta |V\rangle, \text{ where } \theta = -\frac{\pi}{8}, \frac{3\pi}{4}\right\} \quad (1.5)$$

These measurement outcomes correspond to the angles of the HWP's optical axes listed in Table 1.1. The measurements of the experiment are the coincidence counts of photons in the two sensors. They are denoted by $c_{x,a,y,b}$, where x and y encode the chosen bases for A and B respectively, and a and b encode the outcome. From this, the outcome probabilities and therefore the correlations can be estimated.

Table 1.1: Angles of the HWP's optical axes for the chosen measurement operators.

Operator	HWP for Outcome 0	HWP for outcome 1
$\hat{A}_0$	0	45
$\hat{A}_1$	22.5	-22.5
$\hat{B}_0$	11.25	56.25
$\hat{B}_1$	-11.25	33.75

1.2 Waveplate Calibration

Before conducting the actual experiment, the waveplates are calibrated to map the engraved angle on the mount to the true angle of the optical axis. As part of the laboratory experiment, only the HWP_A is calibrated. For this, the HWP_B is fixed, while the HWP_A is rotated in 15° steps, and the coincidences c are recorded. After that, a curve of the shape

$$c(\theta_A) = a \cdot \cos^2(2\theta_A + \delta) + n \quad (1.6)$$

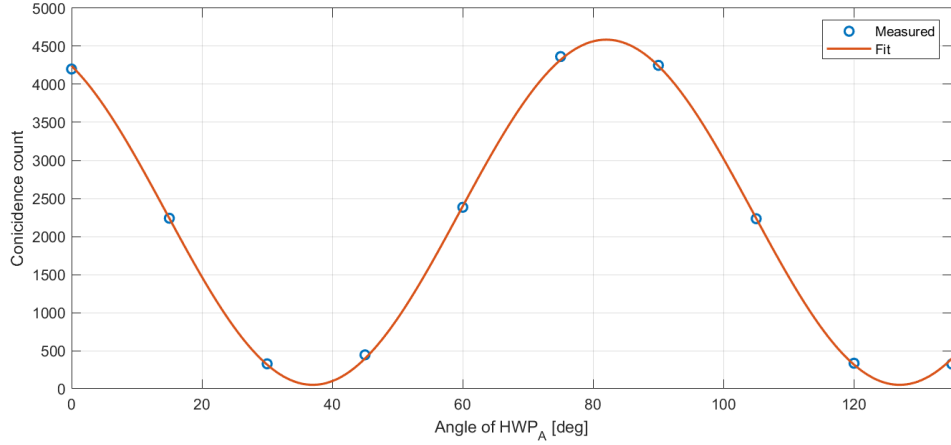


Figure 1.2: *Measured coincidence counts and fitted curve.*

is fit to the data. Here θ_A denotes the engraved angle of the HWP_A, a the maximum possible coincidence count, δ a phase offset between the engraved angle and the optical axis angle, and n an offset due to background noise. Figure 1.2 shows a plot of the recorded data and the fitted curve. The angle for zero coincidence was determined to be $\text{HWP}_A(c_{\min}) = 37^\circ$. The MATLAB code to fit the curve can be found in Appendix A.1.1.

1.3 Data Preparation

The raw data from the measurement unit describes the times at which the photon detectors fired. It is formatted as a two-column matrix with the timestamp of the firing and the channel that fired, and there is one dataset for each combination of measurement outcomes. To get comparable results for each experiment, the datasets are first reduced to a one-second timeframe. Since two detections are never truly simultaneous, a time window has to be chosen to call two events coincident. For this, all time deltas between the two channels are determined and mapped in a histogram. To avoid duplicates, the dataset was separated by channels, and each event in the dataset with less counts was mapped to the closest event of the other channel. For each selected event of the longer dataset, only those events from the shorter channel pointing to it are considered that yield the shortest time deltas. Then the mean μ and standard deviation σ of the histogram bin counts are computed, and a normal distribution is fitted to the data. To improve the fit and reduce the effect of noise and outliers, only matches with a time difference of less than 5 ns were considered. The time window for calling two events coincident is then set to be $\mu \pm 3\sigma$. Figure 1.3 shows the histogram with the fitted and scaled normal distribution and the cut-off for the experiment $x, a, y, b = 0, 0, 0, 0$, and Table 1.2 shows all recorded coincidences. From the histogram, it can be seen that the mean time difference is not zero, but slightly negative. Indeed, this is the case for all measurements. The

mean of the means is $\mu_\mu = -0.229$ ns, with standard deviation $\sigma_\mu = \pm 0.034$ ns likely due to sensor intrinsics. The MATLAB script in Appendix A.1.2 implements the data preparation, plotting all histograms, and computing all coincidences.

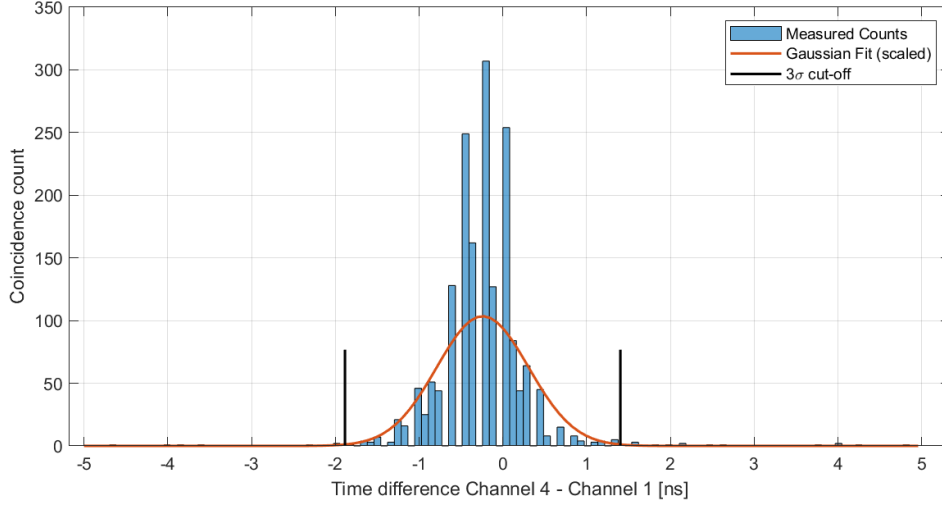


Figure 1.3: *Histogram of time differences for the two recorded channels (experiment $x, a, y, b = 0, 0, 0, 0$)*

Table 1.2: *Recorded coincidence counts for each measurement setting (x, y) , in a one second time window.*

$x = 0, y = 0$			$x = 1, y = 0$		
	$a = 0$	$a = 1$		$a = 0$	$a = 1$
$b = 0$	1733	401	$b = 0$	1790	475
$b = 1$	409	1706	$b = 1$	371	1757
$x = 0, y = 1$			$x = 1, y = 1$		
	$a = 0$	$a = 1$		$a = 0$	$a = 1$
$b = 0$	1967	229	$b = 0$	565	1634
$b = 1$	239	1998	$b = 1$	1624	523

1.4 CHSH Parameter

From the coincidences, the outcome probabilities can be estimated by normalizing over the outcomes for one operator combination:

$$p_{x,a,y,b} = \frac{c_{x,a,y,b}}{\sum_{a',b'} c_{x,a',y,b'}} \quad (1.7)$$

The probabilities are listed in Table 1.3. From that, the correlations can be computed:

$$\langle \hat{A}_x \otimes \hat{B}_y \rangle = \sum_{a,b} \lambda_{x,a} \lambda_{y,b} \cdot p_{x,a,y,b} \quad (1.8)$$

Where $\lambda_{x/y,a/b} \in \{-1, +1\}$ are the eigenvalues of the operators $\hat{A}_x/\hat{B}_y$ corresponding to the outcomes a/b . The correlations are listed in Table 1.4.

Table 1.3: *Estimated probabilities from the coincidences.*

$x = 0, y = 0$			$x = 1, y = 0$		
	$a = 0$	$a = 1$		$a = 0$	$a = 1$
$b = 0$	0.4079	0.0944	$b = 0$	0.4075	0.1081
$b = 1$	0.0963	0.4015	$b = 1$	0.0845	0.4000

$x = 0, y = 1$			$x = 1, y = 1$		
	$a = 0$	$a = 1$		$a = 0$	$a = 1$
$b = 0$	0.4437	0.0517	$b = 0$	0.1300	0.3760
$b = 1$	0.0539	0.4507	$b = 1$	0.3737	0.1203

Table 1.4: *Correlations resulting from the measurements.*

	$x = 0$	$x = 1$
$y = 0$	0.6187	0.6148
$y = 1$	0.7889	-0.4993

Finally, from the correlations, the CHSH parameter can be calculated, and the inequality can be checked:

$$\delta_{CHSH} = \langle \hat{A}_0 \otimes \hat{B}_0 \rangle + \langle \hat{A}_0 \otimes \hat{B}_1 \rangle + \langle \hat{A}_1 \otimes \hat{B}_0 \rangle - \langle \hat{A}_1 \otimes \hat{B}_1 \rangle = 2.5217 > 2 \quad (1.9)$$

So the inequality is violated, proving the entanglement of the photons. The MATLAB script from Appendix A.1.2 also implements the whole computation of the CHSH parameter.

1.5 Error Propagation

The measured coincidences are subject to errors. This can be seen by taking different one-second windows, which would result in different counts. The propagation of the errors on the coincidences to the CHSH parameter can be estimated by the Gaussian error propagation model. This estimates the variance of a function value from the variance of its arguments, using a first-order approximation:

$$\text{Var}[f(x)] = \sum_k \left(\frac{\partial f(x)}{\partial x_k} \right)^2 \text{Var}[x_k] \quad (1.10)$$

The errors on the coincidences are assumed to be Poissonian, and the measurements are assumed to be close to the mean. Therefore, for the variance of the coincidences, it follows:

$$\text{Var}[c_{x,a,y,b}] = c_{x,a,y,b} \quad (1.11)$$

Explicitly expanding the formula for the CHSH parameter in terms of the coincidences yields:

$$\delta_{CHSH}(c) = \sum_{(x,y) \setminus (1,1)} \langle \hat{A}_x \otimes \hat{B}_y \rangle(c) - \langle \hat{A}_1 \otimes \hat{B}_1 \rangle(c) \quad (1.12)$$

$$= \sum_{(x,y) \setminus (1,1)} \sum_{a,b} \lambda_{x,a} \lambda_{y,b} \cdot p_{x,a,y,b}(c) - \sum_{a,b} \lambda_{1,a} \lambda_{1,b} \cdot p_{1,a,1,b}(c) \quad (1.13)$$

$$= \sum_{(x,y) \setminus (1,1)} \sum_{a,b} \lambda_{x,a} \lambda_{y,b} \cdot \frac{c_{x,a,y,b}}{\sum_{a',b'} c_{x,a',y,b'}} - \sum_{a,b} \lambda_{1,a} \lambda_{1,b} \cdot \frac{c_{1,a,1,b}}{\sum_{a',b'} c_{1,a',1,b'}} \quad (1.14)$$

Clearly, each coincidence count only appears in one correlation term, so the partial derivatives can be simplified:

$$\frac{\partial}{\partial c_{x,a,y,b}} \delta_{CHSH}(c) = \frac{\partial}{\partial c_{x,a,y,b}} \langle \hat{A}_x \otimes \hat{B}_y \rangle(c) \quad (1.15)$$

$$= \frac{\partial}{\partial c_{x,a,y,b}} \left(\frac{1}{\sum_{a',b'} c_{x,a',y,b'}} \cdot \sum_{a',b'} \lambda_{x,a'} \lambda_{y,b'} c_{x,a',y,b'} \right) \quad (1.16)$$

$$= - \frac{1}{\left(\sum_{a',b'} c_{x,a',y,b'} \right)^2} \cdot \sum_{a',b'} \lambda_{x,a'} \lambda_{y,b'} c_{x,a',y,b'} + \frac{\lambda_{x,a} \lambda_{y,b}}{\sum_{a',b'} c_{x,a',y,b'}} \quad (1.17)$$

$$= - \frac{\langle \hat{A}_x \otimes \hat{B}_y \rangle}{\sum_{a',b'} c_{x,a',y,b'}} + \frac{\lambda_{x,a} \lambda_{y,b}}{\sum_{a',b'} c_{x,a',y,b'}} \quad (1.18)$$

$$= \frac{\lambda_{x,a} \lambda_{y,b} - \langle \hat{A}_x \otimes \hat{B}_y \rangle}{\sum_{a',b'} c_{x,a',y,b'}} \quad (1.19)$$

Using this to compute the variance of the CHSH parameter, it follows:

$$\text{Var}[\delta_{CHSH}] = 1.993 \cdot 10^{-4} \quad (1.20)$$

Therefore, the 1σ confidence interval is:

$$\delta_{CHSH} = 2.5217 \pm 0.0141 \quad (1.21)$$

The MATLAB code implementing the Gaussian error propagation model is also the one from Appendix A.1.2.

2 Quantum State Tomography

After confirming the successful generation of entangled photon pairs, the next step is to analyze their quantum state. To do this, a quantum state tomography was performed following the procedure outlined in [3]. The hardware setup remains largely the same as in Section 1.1, but this time the QWPs are also used. The density matrix $\hat{\rho}$ of an entangled state is fully characterized by a set of 16 real parameters; to recover them, a set of 16 measurements (tomographic states) with linearly independent projectors is necessary. For each measurement, the optical axes of both HWPs and both QWPs were adjusted, and the coincidence counts c between the two sensors were recorded. The experiment was performed three times: first for the $|\Phi-\rangle$ Bell state; then for the state $|\Phi+\rangle$, which was prepared by rotating one of the HWPs by 45° compared to the settings for $|\Phi-\rangle$, this rotates $|H\rangle \leftrightarrow |V\rangle$, turning $|\Phi-\rangle$ into $|\Phi+\rangle$ up to a global phase; and finally for a partially decoherent version of the $|\Phi-\rangle$ state, obtained by removing the walk-off compensators and opening the iris to increase light flux. The chosen measurement settings and the coincidence counts are listed in Table 2.1, the combinations of measured states are encoded by the ν . For the state $|\Phi+\rangle$ and the decoherent state, only those coincidences, that distinguish them from the mixed state $\rho_{\text{mixed}} = \frac{1}{2}(|H\rangle\langle H| + |V\rangle\langle V|)$ were recorded. The missing counts were extrapolated from the counts recorded for $|\Phi-\rangle$ by normalizing over the first 4 measurements. The extrapolated counts are highlighted in bold font. The MATLAB code for this section can be found in Appendix A.2, it is separated into the main code, and functions implementing subsections.

2.1 Linear Inversion

One method for recovering the density matrix of an entangled state from measurements is by linear inversion. For this, the density matrix is first rewritten in terms of a 16-dimensional column vector r :

$$\hat{\rho} = \sum_{\mu=1}^{16} r_{\mu} \Gamma_{\mu} \quad (2.1)$$

Where Γ_{μ} are the two-qubit Pauli matrices $\hat{\sigma}_i \otimes \hat{\sigma}_j$ ($i, j = 0, 1, 2, 3$). Using this expression in the description of the outcome probabilities, it follows:

$$p_{\nu} = \langle \psi_{\nu} | \hat{\rho} | \psi_{\nu} \rangle \quad (2.2)$$

$$= \sum_{\mu=1}^{16} r_{\mu} \langle \psi_{\nu} | \Gamma_{\mu} | \psi_{\nu} \rangle \quad (2.3)$$

Where $|\psi_{\nu}\rangle$ describes the two-qubit tomographic states. Defining

$$B_{\nu,\mu} := \langle \psi_{\nu} | \Gamma_{\mu} | \psi_{\nu} \rangle \quad (2.4)$$

Table 2.1: *Correlations resulting from the measurements.*

ν	$\hat{A}$	$\hat{B}$	$ \Phi-\rangle$	$ \Phi+\rangle$	$ \Phi-\rangle$ (dec.)
1	$ H\rangle$	$ H\rangle$	3831	3672	18320
2	$ H\rangle$	$ V\rangle$	45	35	290
3	$ V\rangle$	$ V\rangle$	3655	3671	19310
4	$ V\rangle$	$ H\rangle$	71	75	430
5	$ R\rangle$	$ H\rangle$	2218	2175	11189
6	$ R\rangle$	$ V\rangle$	1607	1576	8107
7	$ D\rangle$	$ V\rangle$	1577	1546	7956
8	$ D\rangle$	$ H\rangle$	2219	2176	11194
9	$ D\rangle$	$ R\rangle$	2305	2260	11628
10	$ D\rangle$	$ D\rangle$	408	3253	3908
11	$ R\rangle$	$ D\rangle$	2444	2396	12329
12	$ H\rangle$	$ D\rangle$	1934	1896	9756
13	$ V\rangle$	$ D\rangle$	1588	1557	8011
14	$ V\rangle$	$ L\rangle$	1493	1464	7532
15	$ H\rangle$	$ L\rangle$	2102	2060	10604
16	$ R\rangle$	$ L\rangle$	370	2839	4687

as the elements of a 16×16 matrix, and stacking the probabilities in a 16-dimensional vector p Equation 2.2 can be rewritten as:

$$p = Br \quad (2.5)$$

The choice of tomographic states guarantees the invertability of B , therefore, the vector r can be obtained as:

$$r = B^{-1}p \quad (2.6)$$

Since $p \propto c$, substituting p with the measured coincidences yields a vector that is linearly dependent on r , leading to an unnormalized density matrix $\tilde{\rho}$. The final density matrix is thus:

$$\hat{\rho} = \frac{\tilde{\rho}}{\text{Tr}\{\tilde{\rho}\}} \quad (2.7)$$

This matrix can be shown to be hermitian and has unit trace thanks to the normalization. However, the procedure does not guarantee positive semidefiniteness. It relies only on the data, and therefore, also on noise. Considering the desired density

matrices

$$\hat{\rho}_{\pm} = \frac{1}{2} \begin{bmatrix} 1 & 0 & 0 & \pm 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ \pm 1 & 0 & 0 & 1 \end{bmatrix} \quad (2.8)$$

it is immediate to see that these have two vanishing eigenvalues. Any noise on the coincidences is therefore highly likely to produce negative eigenvalues in the estimated density, making them non-physical. The estimated densities and their eigenvalues (as vectors v) are listed in the following, clearly, all obtained densities are non-physical.

$$\hat{\rho}_{-} = \begin{bmatrix} 0.5039 & -0.0005 + 0.0216i & 0.0353 - 0.0351i & -0.3670 - 0.1283i \\ -0.0005 - 0.0216i & 0.0059 & 0.0117 - 0.0483i & -0.0359 + 0.0320i \\ 0.0353 + 0.0351i & 0.0117 + 0.0483i & 0.0093 & -0.0362 - 0.0487i \\ -0.3670 + 0.1283i & -0.0359 - 0.0320i & -0.0362 + 0.0487i & 0.4808 \end{bmatrix} \quad (2.9)$$

$$\hat{\rho}_{+} = \begin{bmatrix} 0.4927 & 0.0057 + 0.0278i & 0.0405 - 0.0404i & 0.3480 - 0.1283i \\ 0.0057 - 0.0278i & 0.0047 & 0.0623 - 0.0483i & -0.0412 + 0.0372i \\ 0.0405 + 0.0404i & 0.0623 + 0.0483i & 0.0101 & -0.0424 - 0.0549i \\ 0.3480 + 0.1283i & -0.0412 - 0.0372i & -0.0424 + 0.0549i & 0.4926 \end{bmatrix} \quad (2.10)$$

$$\hat{\rho}_{\text{dec.}} = \begin{bmatrix} 0.4777 & 0.0118 + 0.0339i & 0.0474 - 0.0473i & -0.2452 - 0.1283i \\ 0.0118 - 0.0339i & 0.0076 & -0.0136 - 0.0483i & -0.0481 + 0.0441i \\ 0.0474 + 0.0473i & -0.0136 + 0.0483i & 0.0112 & -0.0485 - 0.0610i \\ -0.2452 + 0.1283i & -0.0481 - 0.0441i & -0.0485 + 0.0610i & 0.5035 \end{bmatrix} \quad (2.11)$$

$$v_{-} = \begin{bmatrix} -0.0440 \\ 0.0402 \\ 0.1126 \\ 0.8911 \end{bmatrix} \quad v_{+} = \begin{bmatrix} -0.0900 \\ 0.0266 \\ 0.1979 \\ 0.8655 \end{bmatrix} \quad v_{\text{dec.}} = \begin{bmatrix} -0.0425 \\ 0.0366 \\ 0.2171 \\ 0.7887 \end{bmatrix} \quad (2.12)$$

2.2 Maximum Likelihood Estimation

Another method to retrieve the density matrices from the measurements is by the maximum likelihood estimate. This method ensures physicality by finding the density matrix that is most likely to have produced the observed data through optimization. The starting point is a parametrized form of physical density matrices.

It can be shown that all density matrices can be written in terms of a tridiagonal matrix $\hat{T}$:

$$\hat{T}(t) = \begin{bmatrix} t_1 & 0 & 0 & 0 \\ t_5 + it_6 & t_2 & 0 & 0 \\ t_{11} + it_{12} & t_7 + it_8 & t_3 & 0 \\ t_{15} + it_{16} & t_{13} + it_{14} & t_9 + it_{10} & t_4 \end{bmatrix} \quad (2.13)$$

Parametrized by the vector t , any density matrix can be written as:

$$\hat{\rho}(t) = \frac{\hat{T}^\dagger(t)\hat{T}(t)}{\text{Tr}\{\hat{T}^\dagger(t)\hat{T}(t)\}} \quad (2.14)$$

With this, a loss function can be defined that describes how well a density defined by t describes the measurements:

$$\mathcal{L}(t) = \sum_{\nu=1}^{16} \frac{(p_\nu - \langle \psi_\nu | \hat{\rho}(t) | \psi_\nu \rangle)}{p_\nu} \quad (2.15)$$

$$= \sum_{\nu=1}^{16} \frac{(c_\nu - \langle \psi_\nu | \tilde{\rho}(t) | \psi_\nu \rangle)}{c_\nu} \quad (2.16)$$

where $\tilde{\rho}(t) = \hat{T}^\dagger(t)\hat{T}(t)$ is the unnormalized density matrix. The density that is most likely to have produced the measurements is then defined by:

$$t_{\text{opt}} = \arg \min_t \mathcal{L}(t) \quad (2.17)$$

The obtained densities and their eigenvalues are listed below, all matrices are positive semidefinite, unit trace and hermitian.

$$\hat{\rho}_- = \begin{bmatrix} 0.5042 & 0.0147 + 0.0272i & 0.0389 - 0.0398i & -0.3921 - 0.1224i \\ 0.0147 - 0.0272i & 0.0061 & 0.0010 - 0.0060i & -0.0315 + 0.0267i \\ 0.0389 + 0.0398i & 0.0010 + 0.0060i & 0.0096 & -0.0254 - 0.0445i \\ -0.3921 + 0.1224i & -0.0315 - 0.0267i & -0.0254 + 0.0445i & 0.4802 \end{bmatrix} \quad (2.18)$$

$$\hat{\rho}_+ = \begin{bmatrix} 0.5175 & 0.0088 + 0.0138i & 0.0304 - 0.0184i & 0.3197 - 0.1188i \\ 0.0088 - 0.0138i & 0.0052 & -0.0008 - 0.0076i & -0.0194 + 0.0150i \\ 0.0304 + 0.0184i & -0.0008 + 0.0076i & 0.0115 & -0.0084 - 0.0302i \\ 0.3197 + 0.1188i & -0.0194 - 0.0150i & -0.0084 + 0.0302i & 0.4658 \end{bmatrix} \quad (2.19)$$

$$\hat{\rho}_{\text{dec.}} = \begin{bmatrix} 0.4815 & 0.0159 + 0.0417i & 0.0421 - 0.0498i & -0.2618 - 0.1328i \\ 0.0159 - 0.0417i & 0.0076 & -0.0011 - 0.0080i & -0.0470 + 0.0351i \\ 0.0421 + 0.0498i & -0.0011 + 0.0080i & 0.0122 & -0.0390 - 0.0518i \\ -0.2618 + 0.1328i & -0.0470 - 0.0351i & -0.0390 + 0.0518i & 0.4988 \end{bmatrix} \quad (2.20)$$

$$v_- = \begin{bmatrix} 0.0000 \\ 0.0055 \\ 0.0822 \\ 0.9123 \end{bmatrix} \quad v_+ = \begin{bmatrix} 0.0000 \\ 0.0000 \\ 0.1654 \\ 0.8346 \end{bmatrix} \quad v_{\text{dec.}} = \begin{bmatrix} 0.0000 \\ 0.0003 \\ 0.1988 \\ 0.8009 \end{bmatrix} \quad (2.21)$$

2.3 Quality Measures

There exist many measurements of the quality of the prepared and measured states. Three of these were analyzed for the results; the fidelity, the concurrence and the Von-Neumann entropy. The results are collected in Table 2.2.

Fidelity

The fidelity of an experimentally prepared state is a measure of how close it is to the desired one. It is defined by the measurement probability when the measurement is a projection on the desired state $|\psi\rangle$. Mathematically, it is defined as:

$$F = \langle \psi | \hat{\rho}_{\text{exp.}} | \psi \rangle \quad (2.22)$$

Concurrence

The concurrence is a measure of entanglement of two particles, it is 1 for fully entangled, and 0 for fully separable states. It is defined as:

$$C = \max(0, \sqrt{r_1} - \sqrt{r_2} - \sqrt{r_3} - \sqrt{r_4}) \quad (2.23)$$

where r_i , ($i = 1, 2, 3, 4$) are the sorted right eigenvalues of the matrix

$$\hat{R} = \hat{\rho} \hat{\Sigma} \hat{\rho}^T \hat{\Sigma} \quad (2.24)$$

where $\hat{\Sigma}$ denotes the spin-flip matrix:

$$\hat{\Sigma} = \hat{\sigma}_2 \otimes \hat{\sigma}_2 \quad (2.25)$$

Von-Neuman Entropy

The Von-Neuman entropy is a measure of purity of quantum states, it is 0 for pure states and $\log_2 d = \log_2 4 = 2$ for fully mixed states (here $d = 4$ is the dimension of the Hilbert space). It is defined as:

$$S = - \sum_{i=1}^4 \lambda_i \cdot \log_2 \lambda_i \quad (2.26)$$

This is only defined for positive semidefinite matrices. To still get some information from the matrices obtained by linear inversion, the negative eigenvalues are set to the smallest positive value in MATLAB: `eps=2.2204e-16`.

Table 2.2: *Error measures for the measured states.*

	Fidelity	Concurrence	Von-Neumann Entropy
$\hat{\rho}_{-, \text{linv}}$	0.8594	0.7004	0.6895
$\hat{\rho}_{+, \text{linv}}$	0.8407	0.5558	0.7821
$\hat{\rho}_{\text{deco}, \text{linv}}$	0.7358	0.4937	0.9233
$\hat{\rho}_{-, \text{ml}}$	0.8842	0.8242	0.4582
$\hat{\rho}_{+, \text{ml}}$	0.8113	0.6682	0.6471
$\hat{\rho}_{\text{deco}, \text{ml}}$	0.7519	0.6042	0.7232

As expected, the decoherent state shows lower fidelity and concurrence, and higher Von-Neumann entropy. The fidelity tends to be higher for the densities obtained by maximum likelihood, this is due to the removal of non-physical artifacts in the data. The $|\Phi-\rangle$ state is generally the most entangled state, meaning the preparation was best here.

2.4 Statistical Errors

Since this experiment only involved very few measurements, the presented error measures are not exact. To get a better picture of the true qualities of the state, the statistical errors were analyzed by simulation. For this, the coincidence counts were again assumed to be the means of a Poissonian distribution, and 100 synthetic sets of measurements were generated by taking samples from this distribution. For these samples, density matrices were obtained by linear inversion and maximum likelihood estimation and their error measures were computed. For all error measures the mean and standard deviation were computed, so that the 1σ confidence interval can be obtained. These confidence intervals are listed in Table 2.3.

Table 2.3: *Confidence intervals for the error measures based on statistical simulation.*

	Fidelity	Concurrence	Von-Neumann Entropy
$\hat{\rho}_{-, \text{linv}}$	0.8604 ± 0.0123	0.7031 ± 0.0377	0.6810 ± 0.0571
$\hat{\rho}_{+, \text{linv}}$	0.8394 ± 0.0132	0.5536 ± 0.0352	0.7787 ± 0.0610
$\hat{\rho}_{\text{deco}, \text{linv}}$	0.7364 ± 0.0052	0.4942 ± 0.0181	0.9217 ± 0.0255
$\hat{\rho}_{-, \text{ml}}$	0.8848 ± 0.0087	0.8269 ± 0.0178	0.4499 ± 0.0347
$\hat{\rho}_{+, \text{ml}}$	0.8103 ± 0.0119	0.6667 ± 0.0233	0.6483 ± 0.0271
$\hat{\rho}_{\text{deco}, \text{ml}}$	0.7527 ± 0.0043	0.6045 ± 0.0078	0.7239 ± 0.0088

References

- [1] A. Einstein, B. Podolsky, and N. Rosen, “Can quantum-mechanical description of physical reality be considered complete?”, *Phys. Rev.*, vol. 47, pp. 777–780, 10 May 1935. DOI: 10.1103/PhysRev.47.777.
- [2] J. S. Bell, “On the einstein podolsky rosen paradox”, *Physics Physique Fizika*, vol. 1, pp. 195–200, 3 Nov. 1964. DOI: 10.1103/PhysicsPhysiqueFizika.1.195.
- [3] D. F. V. James, P. G. Kwiat, W. J. Munro, and A. G. White, “Measurement of qubits”, *Phys. Rev. A*, vol. 64, p. 052312, 5 Oct. 2001. DOI: 10.1103/PhysRevA.64.052312.

A Appendix

In the following, the MATLAB scripts and custom functions used are listed.

A.1 CHSH Inequality Violation

This section provides the MATLAB scripts corresponding to Section 1: *CHSH Inequality Violation*.

A.1.1 Waveplate Calibration

Code 1: *waveplate_calibration.m*

```

1  clear
2
3  % Load the data -> .csv file was converted into .mat
4  load("data\Waveplate_A.mat")
5  X = waveplate_A(:,1); % angles in deg
6  Y = waveplate_A(:,2); % coincident counts
7
8
9  % Define fit options and type and perform the fit
10 wp_opt = fitoptions('Method','NonlinearLeastSquares',...
11                    'Lower',[0,0,0],...
12                    'Upper',[Inf,2*pi, min(Y)],...
13                    'StartPoint',[max(Y), 0, 0]);
14
15 wp_typedef = fittype("a*cos(2*x+b).^2+c",...
16                    dependent="y",independent="x",...
17                    coefficients=["a" "b" "c"], options=wp_opt);
18
19 wp_fit = fit(X*pi/180,Y,wp_typedef);
20
21
22 % Compute the fitted curve with minimum and maximum
23 x_val = 0:0.5:135;
24 y_fit = wp_fit.a*cos(2*x_val*pi/180+wp_fit.b).^2+wp_fit.c;
25
26 [~,alpha_min_idx] = min(y_fit);
27 alpha_min = x_val(alpha_min_idx);
28 disp("Minimum of HWP_A at: "+alpha_min)
29
30 % Plot the curve and datapoints
31 f1 = figure(1);
32 plot(X,Y,"o",LineWidth=1.5)
33 hold on

```

```

34 plot(x_val,y_fit,LineWidth=1.5)
35 hold off
36 grid on
37 xlim([0,135])
38 xlabel("Angle of HWP_A [deg]")
39 ylabel("Conicidence count")
40 legend("Measured", "Fit")

```

A.1.2 Data Preparation, CHSH Parameter and Errors

Code 2: *chsh_inequality_violation.m*

```

1  % Script to compute the CHSH parameter and its variance from raw
2  % measurement data. The data is assumed to be stored in file called
3  % "chsh_dataset.mat" in a subdirectory "data" to the root directory of this
4  % file.
5  clear
6
7  %% User Inputs
8
9  % Window to consider counts [s]
10 t_window = 1;
11
12 % Deltas to consider for gaussian fit [ns] (reduce effect of outliers)
13 t_gauss = 5;
14
15 % Device sampling time
16 ts_ps = 80.955; % [ps]
17 ts_ns = ts_ps*1e-3; % [ns]
18
19 % Choice to generate Plots (1:generate, 0:not)
20 generate_plots = 0;
21
22
23 %% Load Data
24 % Load dataset into a struct called "meas"
25 load("data\chsh_dataset.mat")
26 fns = fieldnames(meas);
27
28
29 %% Computation of Coincidences
30 % Preallocate
31 coincidences = zeros(4,4);
32 sample_mean = zeros(4,4);
33

```

```

34
35 for run=1:length(fns)
36     dat = meas.(fns{run});
37
38     % Take middle one second of data
39     samples_per_second = 1e12/ts_ps;
40     middle = dat(1,1) + (dat(end,1)-dat(1,1))/2;
41
42     window = dat((dat(:,1) >= middle-samples_per_second*t_window/2 &...
43                 dat(:,1) <= middle+samples_per_second*t_window/2),:);
44
45     % Split the channels (only save timestamps)
46     ch1 = window(window(:,2) == 1, 1);
47     ch4 = window(window(:,2) == 4, 1);
48
49     % Split by long and short vectors
50     if length(ch1) < length(ch4)
51         short = ch1;
52         long = ch4;
53         short_tag = "Channel 1";
54         long_tag = "Channel 4";
55     else
56         short = ch4;
57         long = ch1;
58         short_tag = "Channel 4";
59         long_tag = "Channel 1";
60     end
61
62     % Preallocate
63     nearest_idx = zeros(length(short),1);
64     delta = zeros(length(short),1);
65
66     % For each short(i), find nearest long(j)
67     for i = 1:length(short)
68         [delta(i), nearest_idx(i)] = min(abs(long - short(i)));
69     end
70
71     % For each chosen long(j), keep only the closest short(i)
72     unique_long = unique(nearest_idx);
73     keep_idx = false(length(short),1);
74
75     for i = 1:length(unique_long)
76         idx = unique_long(i);
77
78         % Find all short events that chose this long event

```

```

79     matching_shorts = find(nearest_idx == idx);
80
81     % Keep only the one with the smallest delta
82     [~, min_pos] = min(delta(matching_shorts));
83     keep_idx(matching_shorts(min_pos)) = true;
84 end
85
86 % Final matched pairs
87 ch_short = short(keep_idx);
88 ch_long = long(nearest_idx(keep_idx));
89
90 % Deltas in [ns]
91 deltas = (ch_short - ch_long)*ts_ns;
92
93 % Filter all events with delta more than 5 ns
94 idx_gauss = find(abs(deltas) <= t_gauss);
95 deltas = deltas(idx_gauss);
96 ch_short = ch_short(idx_gauss);
97 ch_long = ch_long(idx_gauss);
98
99 % Histogram
100 width = ts_ns;
101 [counts, edges] = histcounts(deltas, BinWidth=width);
102 bin_means = edges(1:end-1) + width/2;
103
104 % Sample mean and variance to determine cut off for coincidence window
105 sample_mean(run) = sum(counts.*bin_means)/sum(counts);
106 sample_var = sum(counts.*(bin_means-sample_mean(run)).^2)...
107             /(length(deltas)-1);
108 sigma = sqrt(sample_var);
109
110 % Gaussian fit to the data (plot up to scale factor for visibility)
111 x = -t_gauss:ts_ns:t_gauss;
112 y = normpdf(x,sample_mean(run),sigma);
113
114 % Cut-off for coincidence window
115 cut_off = 3*sigma;
116
117 % Coincidence count
118 idx_cutoff = find(abs(deltas) <= cut_off);
119 coincidence = length(idx_cutoff);
120 coincidences(run) = coincidence;
121
122 % Generate Plots
123 if generate_plots == 1

```

```

124     % Create figure
125     fig_num = "f"+run;
126     fig.(fig_num) = figure(run);
127
128     % Plot the histogram
129     histogram(deltas, BinWidth=ts_ns);
130
131     % Scale the gaussian
132     cdf = normcdf([-t_gauss,t_gauss],sample_mean(run),sigma);
133     norm_fact = width*sum(counts)*(cdf(2)-cdf(1));
134     hold on
135     plot(x,y*norm_fact, LineWidth=1.5)
136
137     % Plot the cut-off values as vertical lines
138     bar = max(counts)/4;
139     plot([1,1]*cut_off+sample_mean(run),[0,bar],'k',LineWidth=1.5)
140     plot([1,1]*-cut_off+sample_mean(run),[0,bar],'k',LineWidth=1.5)
141     hold off
142
143     grid on
144     xlabel("Time difference "+short_tag+" - "+long_tag+" [ns]");
145     ylabel("Coincidence count")
146     legend("Measured Counts","Gaussian Fit (scaled)","3\sigma cut-off")
147 end
148 end
149
150 % Mean of means and standard deviation
151 sample_mean = reshape(sample_mean,[16,1]);
152 mean_of_means = mean(sample_mean);
153 std_of_means = sqrt(var(sample_mean));
154
155
156 %% Compute Probabilities and Correlations
157 % Total coincidence counts for each combination of bases
158 total = zeros(4,4);
159 total(1:2,1:2) = sum(coincidences(1:2,1:2), "all");
160 total(3:4,1:2) = sum(coincidences(3:4,1:2), "all");
161 total(1:2,3:4) = sum(coincidences(1:2,3:4), "all");
162 total(3:4,3:4) = sum(coincidences(3:4,3:4), "all");
163
164 % Probabilities
165 probabilities = coincidences./total;
166
167 % Correlations
168 eval_comb = zeros(4,4); % Combinations of all eigenvalues

```

```
169 for i=1:4
170     for j=1:4
171         eval_comb(i,j) = (-1)^(i+j);
172     end
173 end
174 correlations = zeros(2,2);
175 prob_times_evals = probabilities.*eval_comb;
176 correlations(1,1) = sum(prob_times_evals(1:2,1:2), "all");
177 correlations(2,1) = sum(prob_times_evals(3:4,1:2), "all");
178 correlations(1,2) = sum(prob_times_evals(1:2,3:4), "all");
179 correlations(2,2) = sum(prob_times_evals(3:4,3:4), "all");
180
181 % CHSH Parameter
182 chsh = abs(sum(correlations(1:3))-correlations(4));
183
184
185 %% Gaussian Method of Linear Error Propagation
186 % Compute the partial derivatives
187 for i=1:2
188     for j=1:2
189         % expand the correlations matrix to be 4x4 for easier computation
190         correlations_4x4(i*2-1:i*2,j*2-1:j*2)=ones(2,2)*correlations(i,j);
191     end
192 end
193 partials = 1./total.*(eval_comb-correlations_4x4);
194
195 % Summands for method
196 summands = partials.^2 .*coincidences;
197
198 % Variance of chsh parameter
199 var_chsh = sum(summands, "all") - 2*sum(summands(3:4,3:4), "all");
200
201 % Standard deviation
202 sigma_chsh = sqrt(var_chsh);
```

A.2 Quantum State Tomography

This section provides the MATLAB script and the used functions corresponding to Section 2: *Quantum State Tomography*.

A.2.1 Script

Code 3: *quantum_state_tomography.m*

```

1  % Script to perform a quantum state tomography based on coincidence count
2  % data. The data is assumed to be stored in a file called "tomography.mat"
3  % in a subdirectory "data" to the root directory of this file. It invokes a
4  % set of custom functions, that are assumed to be stored in a subdirectory
5  % called "functions".
6  clear
7  addpath('functions')
8
9  %% Load Data from the Experiment
10 load("data\tomography.mat")
11
12 % Extrapolate the missing data from coinc_minus to plus and decoherence
13 scale_plus = sum(coinc_plus(1:4))/sum(coinc_minus(1:4));
14 scale_deco = sum(coinc_deco(1:4))/sum(coinc_minus(1:4));
15
16 coinc_plus([5:9, 11:15]) = coinc_minus([5:9, 11:15])*scale_plus;
17 coinc_deco([5:9, 11:15]) = coinc_minus([5:9, 11:15])*scale_deco;
18
19
20 %% Define the measurements
21 % Normalized basis vectors
22 H = [1;0];
23 V = [0;1];
24 D = 1/sqrt(2)*(H+V);
25 R = 1/sqrt(2)*(H+1i*V);
26 L = 1/sqrt(2)*(H-1i*V);
27
28 % States to be measured
29 Phi_m = 1/sqrt(2)*(kron(H,H) - kron(V,V));
30 Phi_p = 1/sqrt(2)*(kron(H,H) + kron(V,V));
31
32 % Outcome pairs in correct order
33 Psi = {{H,H}; {H,V}; {V,V}; {V,H}; {R,H}; {R,V}; {D,V}; {D,H};
34       {D,R}; {D,D}; {R,D}; {H,D}; {V,D}; {V,L}; {H,L}; {R,L}};
35
36 for nu=1:16
37     Psi{nu} = kron(Psi{nu}{1}, Psi{nu}{2});

```

```

38 end
39
40
41 %% Compute the densities for |Psi+>, |Psi-> and the Decoherent State
42 % By linear inversion
43 % Define the Pauli matrices
44 sigma_0 = eye(2,2);
45 sigma_1 = [0, 1; 1, 0];
46 sigma_2 = [0, -1i; 1i, 0];
47 sigma_3 = [1, 0; 0, -1];
48 pauli_matrices = {sigma_0, sigma_1, sigma_2, sigma_3};
49
50 % Define the Gamma matrices
51 Gamma = cell(16,1);
52 mu = 1;
53 for i=1:4
54     for j=1:4
55         Gamma{mu} = kron(pauli_matrices{i}, pauli_matrices{j});
56         mu = mu+1;
57     end
58 end
59
60 % Compute the density
61 rho_minus_linv = density_from_linear_inversion(coinc_minus, Psi, Gamma);
62 rho_plus_linv = density_from_linear_inversion(coinc_plus, Psi, Gamma);
63 rho_deco_linv = density_from_linear_inversion(coinc_deco, Psi, Gamma);
64
65
66 % By maximum likelihood
67 rho_minus_ml = density_from_maximum_likelihood(coinc_minus, Psi);
68 rho_plus_ml = density_from_maximum_likelihood(coinc_plus, Psi);
69 rho_deco_ml = density_from_maximum_likelihood(coinc_deco, Psi);
70
71
72 %% Error Analysis
73 % For linear inversion
74 errors_minus_linv = error_analysis(rho_minus_linv, Phi_m);
75 errors_plus_linv = error_analysis(rho_plus_linv, Phi_p);
76 errors_deco_linv = error_analysis(rho_deco_linv, Phi_m);
77
78 % For maximum likelihood
79 errors_minus_ml = error_analysis(rho_minus_ml, Phi_m);
80 errors_plus_ml = error_analysis(rho_plus_ml, Phi_p);
81 errors_deco_ml = error_analysis(rho_deco_ml, Phi_m);
82

```

```

83
84
85 %% Statistical Errors by Simulation
86 % Preallocate
87 errors_minus_linv_s = zeros(100,3);
88 errors_minus_ml_s = zeros(100,3);
89 errors_plus_linv_s = zeros(100,3);
90 errors_plus_ml_s = zeros(100,3);
91 errors_deco_linv_s = zeros(100,3);
92 errors_deco_ml_s = zeros(100,3);
93
94 for i=1:100
95     % Synthetic measurements
96     meas_minus = poissrnd(coinc_minus);
97     meas_plus = poissrnd(coinc_plus);
98     meas_deco = poissrnd(coinc_deco);
99
100    % Get linear inversion rho
101    rho_minus_linv_s = density_from_linear_inversion(meas_minus,Psi,Gamma);
102    rho_plus_linv_s = density_from_linear_inversion(meas_plus,Psi,Gamma);
103    rho_deco_linv_s = density_from_linear_inversion(meas_deco,Psi,Gamma);
104
105    % Get max likelihood rho
106    rho_minus_ml_sim = density_from_maximum_likelihood(meas_minus,Psi);
107    rho_plus_ml_sim = density_from_maximum_likelihood(meas_plus,Psi);
108    rho_deco_ml_sim = density_from_maximum_likelihood(meas_deco,Psi);
109
110    % Errors (1 specifies that the Von-Neuman entropy is not calculated)
111    errors_minus_linv_s(i,:) = error_analysis(rho_minus_linv_s,Phi_m);
112    errors_minus_ml_s(i,:) = error_analysis(rho_minus_ml_sim,Phi_m);
113    errors_plus_linv_s(i,:) = error_analysis(rho_plus_linv_s,Phi_p);
114    errors_plus_ml_s(i,:) = error_analysis(rho_plus_ml_sim,Phi_p);
115    errors_deco_linv_s(i,:) = error_analysis(rho_deco_linv_s,Phi_m);
116    errors_deco_ml_s(i,:) = error_analysis(rho_deco_ml_sim,Phi_m);
117 end
118
119 % Mean and standard deviation of the generated samples
120 errors_minus_linv_mean = mean(errors_minus_linv_s);
121 errors_minus_ml_mean = mean(errors_minus_ml_s);
122 errors_plus_linv_mean = mean(errors_plus_linv_s);
123 errors_plus_ml_mean = mean(errors_plus_ml_s);
124 errors_deco_linv_mean = mean(errors_deco_linv_s);
125 errors_deco_ml_mean = mean(errors_deco_ml_s);
126
127 errors_minus_linv_std = sqrt(var(errors_minus_linv_s));

```

```

128 errors_minus_ml_std = sqrt(var(errors_minus_ml_s));
129 errors_plus_linv_std = sqrt(var(errors_plus_linv_s));
130 errors_plus_ml_std = sqrt(var(errors_plus_ml_s));
131 errors_deco_linv_std = sqrt(var(errors_deco_linv_s));
132 errors_deco_ml_std = sqrt(var(errors_deco_ml_s));

```

A.2.2 Functions

Code 4: *density_from_linear_inversion.m*

```

1  function rho = density_from_linear_inversion(coinc,Psi,Gamma)
2  % DENSITY_FROM_LINEAR_INVERSION compues the density matrix of a 2 qubit
3  % state by linear inversion. The resulting density matrix is not guaranteed
4  % to be physical.
5  %
6  % coinc - measured coincidences as a vector
7  % Psi    - the measured states as a cell array of vectors
8  % Gamma  - the 16 tensor products of the pauli matrices as a cellarray
9
10 % Compute the B matrix
11 B = zeros(16,16);
12 for nu=1:16
13     for mu=1:16
14         B(nu,mu) = Psi{nu}' *Gamma{mu}* Psi{nu};
15     end
16 end
17
18 % Compute the r vector
19 r = B\coinc;
20
21 % Compute the unnormalized density
22 rho = zeros(4, 4); % 2-qubit system -> 4x4 matrix
23 for mu = 1:16
24     rho = rho + r(mu)*Gamma{mu};
25 end
26
27 % Normalize the density
28 rho = rho/trace(rho);
29 end
30

```

Code 5: *density_from_maximum_likelihood.m*

```

1  function rho = density_from_maximum_likelihood(coinc, Psi)
2  % DENSITY_FROM_MAXIMUM_LIKELIHOOD compues the density matrix of a 2 qubit
3  % state by maximum likelihood. The resulting density matrix is the one,
4  % that best describes the data while beig physical.
5  %
6  % coinc - measured coincidences as a vector
7  % Psi    - the measured states as a cell array of vectors
8
9  % Initial guess for t
10 t0 = ones(16,1);
11
12 % Optimizer options
13 options = optimoptions('fminunc','Display','off',...
14                        MaxFunctionEvaluations=1e4); % increase max evals
15
16 % Find optimal t
17 t_opt = fminunc(@(t)squared_loss(t,Psi,coinc), t0, options);
18
19 % Unnormalized density
20 rho = unnormal_density(t_opt);
21
22 % Normalized density
23 rho = rho/trace(rho);
24 end
25

```

Code 6: *squared_loss.m*

```

1  function loss = squared_loss(t, Psi, C)
2  % SQUARED_LOSS computes the squared loss of a proposed density matrix wrt
3  % the measurement coincidences
4  %
5  % t    - proposed parameters in denisty matrix
6  % Psi  - measured states, as a cell array
7  % C    - coincidences
8
9  rho = unnormal_density(t);
10
11 loss = 0;
12 for nu=1:length(C)
13     % real for numerical stability => might get 1i*1e-16 => minim. error

```

```
14     loss = loss + real((C(nu) - Psi{nu}' *rho* Psi{nu})^2 / C(nu));
15 end
```

Code 7: *unnorml_density.m*

```
1 function rho = unnorml_density(t)
2 % UNNORML_DENSITY computes an unnormalized density matrix based on 16
3 % parameters stored in the input vector t
4
5 T = [t(1),          0,          0,          0;
6      t(5)+1i*t(6),  t(2),          0,          0;
7      t(11)+1i*t(12), t(7)+1i*t(8),  t(3),          0;
8      t(15)+1i*t(16), t(13)+1i*t(14), t(9)+1i*t(10), t(4)];
9
10 rho = T'*T;
11 end
```
